

Supplementary Material 1: Scalability benchmark against xcms

Instantiation time and memory of two Chromatograms backends (*ChromBackendMemory*, *ChromBackendSpectra*) compared with the xcms *MChromatograms* implementation

Philippine Louail

Laurent Gatto

Sebastian Gibb

Johannes Rainer

2026-06-03

Table of contents

Motivation	1
System Information	2
1. Load Raw Data	3
2. Define EIC Extraction Targets	3
3. Benchmark: Object Instantiation	4
Quantifying the super-linearity directly from the timings	8
4. Benchmark: Object Size (RAM Footprint)	10
5. Benchmark: Data Access (Intensity Extraction)	11
6. Benchmark: Computation via Lazy Processing Queue	12
7. Scaling: Multiple Files	16
Quantifying scaling directly (multi-file)	18
8. Summary Table	20
Discussion	21
Key results for manuscript	21
Session Info	22

Supplementary Material for:

Chromatograms: A modular infrastructure for scalable chromatographic data handling in R

Philippine Louail^{1,2,*}, Laurent Gatto³, Sebastian Gibb⁴, Johannes Rainer¹

¹ Institute for Biomedicine, Eurac Research, Bolzano, Italy ² Chair for Bioinformatics, Friedrich-Schiller-University Jena, 07743 Jena, Germany ³ Computational Biology and Bioinformatics, de Duve Institute, UCLouvain, Brussels, Belgium ⁴ Department of Anaesthesiology and Intensive Care, University Medicine Greifswald, Greifswald, Germany

* Corresponding author: philippine.louail@outlook.com

Motivation

Established mass spectrometry tools in R, such as *xcms*, materialise all chromatographic data into memory at object-creation time. **Chromatograms** takes a different approach: a modular backend system decouples the container from the storage layer, so users can choose between:

Backend	Behaviour	Use case
<i>ChromBackendMemory</i>	All data in RAM	Small to medium datasets, fast repeated access
<i>ChromBackendSpectra</i>	On-the-fly extraction from a Spectra object	Large datasets, low memory footprint
<i>ChromBackendMzR</i>	On-disk via <i>mzR</i>	Very large experiments, minimal RAM

This document compares the three backends against the *xcms* approach on real mzML data, measuring object **instantiation time** and **RAM footprint** as the number of EIC targets and source files grows. Timings are reported in seconds (s); memory in megabytes (MB).

```
library(Chromatograms)
library(Spectra)
library(MsBackendMetaboLights)
library(xcms)
library(MsExperiment)
library(bench)
library(peakRAM)
library(microbenchmark)
library(dplyr)
library(ggplot2)
library(tidyr)
library(BiocParallel)

# Use serial processing for reproducible benchmarks
register(SerialParam())

# Consistent colour palette for all plots
method_colours <- c(
  "ChromBackendSpectra" = "#1B9E77",
  "ChromBackendMemory"  = "#D95F02",
  "xcms MChromatograms" = "#7570B3"
)

scale_colour_methods <- function(...) {
  scale_colour_manual(values = method_colours, ...)
}

scale_fill_methods <- function(...) {
  scale_fill_manual(values = method_colours, ...)
}
```

System Information

```
sys_info <- data.frame(  
  Property = c("CPU Cores", "R Version", "Platform"),  
  Value = c(as.character(parallel::detectCores()),  
            R.version$version.string, R.version$platform)  
)  
for (pkg in c("Chromatograms", "Spectra", "xcms")) {  
  if (requireNamespace(pkg, quietly = TRUE))  
    sys_info <- rbind(sys_info, data.frame(  
      Property = paste(pkg, "version"),  
      Value = as.character(packageVersion(pkg)))  
    )  
}  
knitr::kable(sys_info, col.names = c("Property", "Value"),  
             caption = "System information")
```

Table 1: System information

Property	Value
CPU Cores	48
R Version	R version 4.6.0 (2026-04-24)
Platform	x86_64-pc-linux-gnu
Chromatograms version	1.2.0
Spectra version	1.22.0
xcms version	4.10.0

1. Load Raw Data

We use the LC-MS files from the MetaboLights study MTBLS8735.

```
# Load public data from MetaboLights (MTBLS8735)  
sps <- Spectra(  
  source = MsBackendMetaboLights(),  
  mtblsId = "MTBLS8735",  
  assayName = paste0("a_MTBLS8735_LC-MS_positive_",  
                     "hilic_metabolite_profiling.txt"),  
  filePattern = ".mzML"  
)  
  
# Get individual file paths from the cached data  
mzml_files <- unique(dataOrigin(sps))  
  
knitr::kable(  
  data.frame(Property = c("mzML files", "Total spectra"),  
            Value = c(length(mzml_files), length(sps))),  
  col.names = c("Property", "Value"),  
  caption = "Raw data summary"  
)
```

Table 2: Raw data summary

Property	Value
mzML files	10
Total spectra	17210

2. Define EIC Extraction Targets

We generate up to **1 000 EIC** regions from a single file to benchmark all three approaches across **1, 10, 100, 500, and 1 000** targets, studying how instantiation time and memory scale with the number of extraction targets.

```
# Get the RT and m/z range of the data
rt_range <- range(runtime(sps))
mz_range <- range(unlist(mz(sps), use.names = FALSE))

# Generate EIC targets spanning the data range
set.seed(123)
n_eics_max <- 1000L
n_eics_vec <- c(1L, 10L, 100L, 500L, 1000L)

target_mz <- runif(n_eics_max, mz_range[1] + 10, mz_range[2] - 10)
mz_tol_da <- 0.01 # ± 0.01 Da window

eic_table <- data.frame(
  mzMin      = target_mz - mz_tol_da,
  mzMax      = target_mz + mz_tol_da,
  rtMin      = rep(rt_range[1], n_eics_max),
  rtMax      = rep(rt_range[2], n_eics_max),
  msLevel    = 1L,
  dataOrigin = rep(unique(dataOrigin(sps))[1], n_eics_max)
)

knitr::kable(
  data.frame(
    Property = c("RT range (s)", "m/z range", "EIC targets (max)",
                 "Instantiation sizes tested"),
    Value    = c(paste(round(rt_range, 1), collapse = " - "),
                 paste(round(mz_range, 2), collapse = " - "),
                 n_eics_max,
                 paste(n_eics_vec, collapse = ", "))
  ),
  col.names = c("Property", "Value"),
  caption = "Data range and EIC target summary"
)
```

Table 3: Data range and EIC target summary

Property	Value
RT range (s)	0.3 – 480.2
m/z range	49.99 – 1000.02
EIC targets (max)	1000
Instantiation sizes tested	1, 10, 100, 500, 1000

3. Benchmark: Object Instantiation

How does object creation scale with the number of EIC targets? We compare the three backends - *ChromBackendSpectra*, *ChromBackendMemory* (via *setBackend()*), and the *xcms chromatogram()* function - across 1, 10, 100, 500, and 1 000 targets.

ChromBackendSpectra stores only metadata (m/z and RT boundaries) and a reference to the *Spectra* object - no data are extracted at this stage. *ChromBackendMemory* is created from a *ChromBackendSpectra* via *setBackend()*, which iterates through each chromatogram and extracts the intensity data into RAM. *xcms' chromatogram()* extracts all data immediately at creation time, like *ChromBackendMemory*.

```
# xcms requires an MsExperiment with linked sample data
od <- MsExperiment()
spectra(od) <- sps
abs_files <- normalizePath(mzml_files)
sampleData(od) <- S4Vectors::DataFrame(
  sample = seq_along(abs_files),
  file   = abs_files
)
od <- linkSampleData(od,
  with = "sampleData.file = spectra.dataOrigin"
)
```

```
instantiation_results <- lapply(n_eics_vec, function(n_eic) {
  eic_sub <- eic_table[seq_len(n_eic), , drop = FALSE]

  bench_res <- bench::mark(
    ChromBackendSpectra = {
      Chromatograms(sps, chromData = eic_sub)
    },
    ChromBackendMemory = {
      setBackend(Chromatograms(sps, chromData = eic_sub),
        ChromBackendMemory())
    },
    `xcms MChromatograms` = {
      chromatogram(
        od[1],
        mz = cbind(eic_sub$mzMin, eic_sub$mzMax),
        rt = cbind(eic_sub$rtMin, eic_sub$rtMax),
```

```

      msLevel = 1L
    )
  },
  iterations = 5,
  check = FALSE
)

# Peak memory: bench::mark only reports cumulative profmem allocations,
# which over-counts because allocate-then-free traffic is included.
# peakRAM measures the actual peak RAM held during a single execution.
peak_res <- peakRAM::peakRAM(
  { Chromatograms(sps, chromData = eic_sub) },
  { setBackend(Chromatograms(sps, chromData = eic_sub),
    ChromBackendMemory()) },
  { chromatogram(
    od[1],
    mz = cbind(eic_sub$mzMin, eic_sub$mzMax),
    rt = cbind(eic_sub$rtMin, eic_sub$rtMax),
    msLevel = 1L
  ) }
)

peak_lookup <- setNames(
  peak_res$Peak_RAM_Used_MiB,
  c("ChromBackendSpectra", "ChromBackendMemory", "xcms MChromatograms")
)

bench_res |>
  mutate(
    method      = as.character(expression),
    n_eics       = n_eic,
    median_s     = as.numeric(median),
    sd_s         = sapply(time, function(t) sd(as.numeric(t))),
    peak_mb      = peak_lookup[as.character(expression)]
  ) |>
  select(method, n_eics, median_s, sd_s, peak_mb)
})

instantiation_df <- bind_rows(instantiation_results)

ggplot(instantiation_df,
  aes(x = factor(n_eics, levels = n_eics_vec), y = median_s, colour = method, group = method),
  geom_errorbar(aes(ymin = pmax(median_s - sd_s, 1e-9), ymax = median_s + sd_s),
    width = 0.2, linewidth = 0.6, alpha = 0.6) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_y_log10() +
  scale_colour_methods() +

```

```

labs(
  x = "Number of EIC Targets",
  y = "Median Time (seconds, log scale)",
  colour = "Method",
  title = "Object Instantiation Time vs Number of EICs"
) +
theme_minimal(base_size = 13)
ggplot(instantiation_df,
  aes(x = factor(n_eics, levels = n_eics_vec), y = peak_mb, colour = method, group = method)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_y_log10() +
  scale_colour_methods() +
  labs(
    x = "Number of EIC Targets",
    y = "Peak RAM (MB, log scale)",
    colour = "Method",
    title = "Peak Memory vs Number of EICs"
  ) +
  theme_minimal(base_size = 13)

```

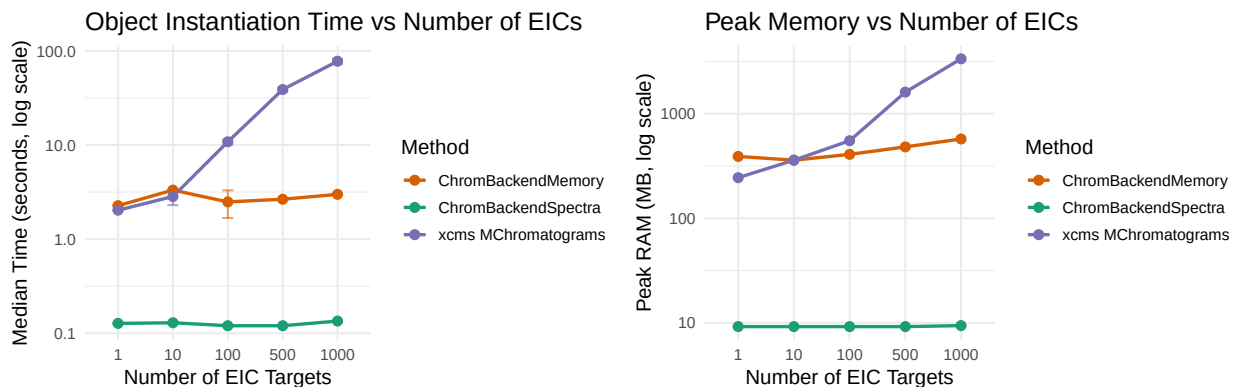


Figure 1: Object instantiation time across EIC target counts. Points and error bars show the median and standard deviation across five iterations.

Figure 2: Peak RAM held during instantiation across EIC target counts. Points and error bars show the median and standard deviation across five iterations.

```

knitr::kable(
  instantiation_df |>
    mutate(
      median_ms = round(median_s * 1e3, 2),
      peak_mb = round(peak_mb, 2)
    ) |>
    select(method, n_eics, median_ms, peak_mb),
  col.names = c("Method", "EICs", "Median Time (ms)", "Peak RAM (MB)"),
  caption = "Object instantiation benchmark across EIC counts"
)

```

Table 4: Object instantiation benchmark across EIC counts

Method	EICs	Median Time (ms)	Peak RAM (MB)
ChromBackendSpectra	1	127.34	9.2
ChromBackendMemory	1	2267.87	390.3
xcms MChromatograms	1	2029.63	244.5
ChromBackendSpectra	10	129.31	9.2
ChromBackendMemory	10	3319.69	359.5
xcms MChromatograms	10	2836.18	359.5
ChromBackendSpectra	100	120.28	9.2
ChromBackendMemory	100	2486.90	408.6
xcms MChromatograms	100	10816.60	551.9
ChromBackendSpectra	500	120.27	9.2
ChromBackendMemory	500	2653.53	481.8
xcms MChromatograms	500	38848.54	1609.4
ChromBackendSpectra	1000	134.82	9.4
ChromBackendMemory	1000	2991.22	573.5
xcms MChromatograms	1000	77615.08	3351.4

The figures and table above show how instantiation time and peak memory scale with target count for each approach. The **Peak RAM (MB)** column reports the maximum resident RAM held during a single execution of each method, measured via *peakRAM()*. Unlike cumulative allocation (`bench::mark(memory = TRUE)$mem_alloc`), which sums every allocation including short-lived buffers that are freed before the call returns, the peak captures the actual high-water mark.

Quantifying the super-linearity directly from the timings

Rather than fitting a power-law, we quantify how strongly each method departs from linear scaling by comparing the time at the **largest** EIC target count to the time at the **smallest** meaningful count (excluding $n = 1$, which is dominated by fixed per-call overhead). If a method scales **strictly linearly**, the ratio $t(n_{\max})/t(n_{\min})$ should equal $n_{\max}/n_{\min}$. A ratio larger than this linear expectation reveals **super-linear** scaling; a ratio close to 1 (independent of n) reveals **flat / essentially constant** behaviour.

```
# Drop n=1 (dominated by fixed overhead) and any zero medians.
scaling_subset <- instantiation_df |>
  filter(n_eics > 1L, median_s > 0)

n_min <- min(scaling_subset$n_eics)
n_max <- max(scaling_subset$n_eics)
n_ratio_expected <- n_max / n_min      # linear-scaling expectation

scaling_ratios <- scaling_subset |>
  filter(n_eics %in% c(n_min, n_max)) |>
  select(method, n_eics, median_s) |>
  tidyr::pivot_wider(names_from = n_eics, values_from = median_s,
                     names_prefix = "t_n") |>
```



```

rename(t_min = paste0("t_n", n_min),
       t_max = paste0("t_n", n_max)) |>
mutate(
  time_ratio      = t_max / t_min,          # observed
  linear_ratio    = n_ratio_expected,      # expected if linear
  super_linear_factor = time_ratio / linear_ratio # >1 = super-linear
)

# Expose values for inline use in the manuscript text.
ratio_mem <- scaling_ratios$time_ratio[
  scaling_ratios$method == "ChromBackendMemory"]
ratio_xcms <- scaling_ratios$time_ratio[
  scaling_ratios$method == "xcms MChromatograms"]
ratio_spec <- scaling_ratios$time_ratio[
  scaling_ratios$method == "ChromBackendSpectra"]

sl_mem <- scaling_ratios$super_linear_factor[
  scaling_ratios$method == "ChromBackendMemory"]
sl_xcms <- scaling_ratios$super_linear_factor[
  scaling_ratios$method == "xcms MChromatograms"]

knitr::kable(
  scaling_ratios |>
  mutate(across(c(t_min, t_max), \(x) round(x, 3)),
         across(c(time_ratio, super_linear_factor), \(x) round(x, 2))),
  col.names = c("Method",
                 paste0("t at n=", n_min, " (s)"),
                 paste0("t at n=", n_max, " (s)"),
                 "Observed ratio",
                 "Linear expectation",
                 "Super-linear factor"),
  caption = paste0(
    "Direct comparison of instantiation time at the smallest vs largest ",
    "EIC count. The 'super-linear factor' is the observed ratio divided ",
    "by the linear expectation of ", n_ratio_expected, "x; ",
    "1.0 = strictly linear, >1.0 = super-linear, <1.0 = sub-linear/flat."
  )
)

```

Table 5: Direct comparison of instantiation time at the smallest vs largest EIC count. The ‘super-linear factor’ is the observed ratio divided by the linear expectation of 100x; 1.0 = strictly linear, >1.0 = super-linear, <1.0 = sub-linear/flat.

Method	t at n=10 (s)	t at n=1000 (s)	Observed ratio	Linear expectation	Super-linear factor
ChromBackendSpectra	0.129	0.135	1.04	100	0.01
ChromBackendMemory	3.320	2.991	0.90	100	0.01

Method	t at n=10 (s)	t at n=1000 (s)	Observed ratio	Linear expectation	Super-linear factor
xcms MChromatograms	2.836	77.615	27.37	100	0.27

The two key numbers for the manuscript text are derived directly from the linear-axis figure:

- **ChromBackendSpectra**: time barely changes from $n = 10$ to $n = 1000$ (observed ratio $\approx 1.04\times$ while linear scaling would give $100\times$) - i.e. **essentially constant**, because only metadata are stored at instantiation.
- **ChromBackendMemory**: observed ratio $\approx 0.9\times$ - i.e. **roughly linear** with the number of EIC targets (super-linear factor ≈ 0.01).
- **xcms MChromatograms**: observed ratio $\approx 27.37\times$ - **super-linear**, i.e. $0.3\times$ the increase that strict linear scaling would predict for the same range.

```
# Create 100-EIC objects for subsequent sections (Sections 4-6)
eic_100 <- eic_table[seq_len(100L), , drop = FALSE]
chr_spectra <- Chromatograms(sps, chromData = eic_100)
chr_mem <- setBackend(chr_spectra, ChromBackendMemory())
chr_xcms <- chromatogram(
  od[1],
  mz = cbind(eic_100$mzMin, eic_100$mzMax),
  rt = cbind(eic_100$rtMin, eic_100$rtMax),
  msLevel = 1L
)
```

4. Benchmark: Object Size (RAM Footprint)

After creating the objects, how much RAM does each occupy? `object.size()` reports the total bytes of each R object. For *ChromBackendSpectra* this includes a full in-process copy of the *Spectra* object (i.e. all raw spectra from all files), so its reported size is larger than the other backends. When the original *sps* is no longer needed after creating *chr_spectra*, calling `rm(sps)` avoids holding two copies of the spectral data in memory.

```
# Total sizes (object.size includes all referenced objects)
total_spectra <- as.numeric(object.size(chr_spectra))
total_mem <- as.numeric(object.size(chr_mem))
total_xcms <- as.numeric(object.size(chr_xcms))

# Embedded Spectra copy and wrapper overhead (used in inline text below)
sps_size <- as.numeric(object.size(sps))
wrapper_overhead <- total_spectra - sps_size

size_df <- data.frame(
  Object = c("ChromBackendSpectra", "ChromBackendMemory", "xcms MChromatograms"),
  Total_Mb = round(c(total_spectra, total_mem, total_xcms) / 1024^2, 2)
)
```

```
knitr::kable(
  size_df,
  col.names = c("Object", "Total RAM (MB)"),
  caption = "RAM footprint after object creation"
)
```

Table 6: RAM footprint after object creation

Object	Total RAM (MB)
ChromBackendSpectra	3.49
ChromBackendMemory	2.72
xcms MChromatograms	2.99

```
ggplot(size_df, aes(x = reorder(Object, Total_Mb), y = Total_Mb, fill = Object)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0(Total_Mb, " MB")), hjust = -0.1, size = 4) +
  coord_flip() +
  scale_fill_methods() +
  labs(x = NULL, y = "Size (MB)",
       title = "RAM Footprint: Chromatogram Containers") +
  theme_minimal(base_size = 13) +
  expand_limits(y = max(size_df$Total_Mb) * 1.3)
```

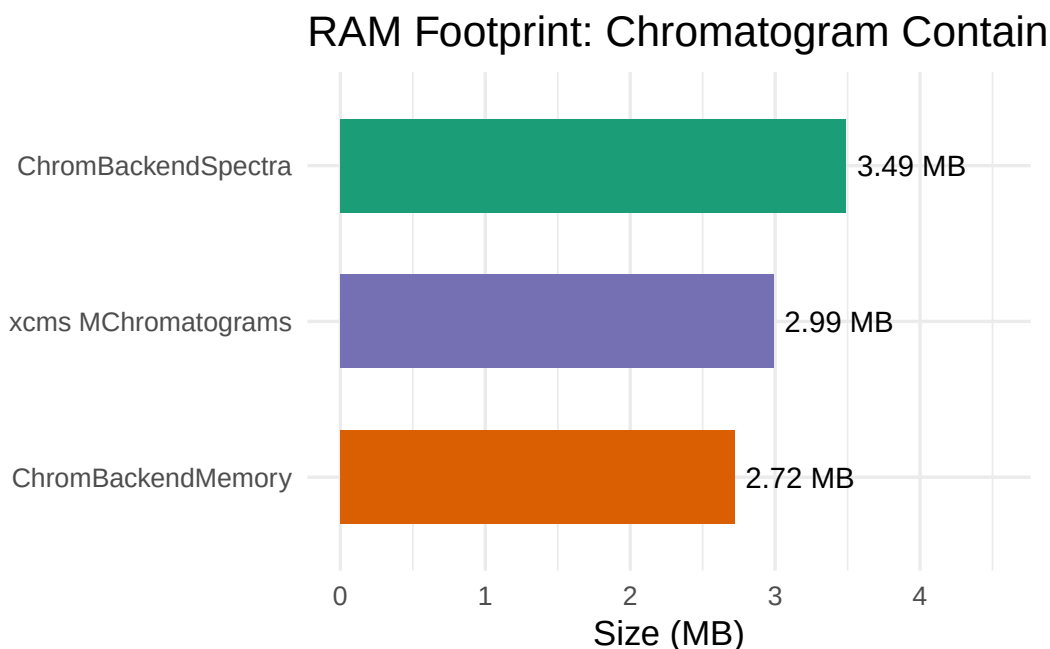


Figure 3: Total RAM footprint of each chromatogram container

ChromBackendSpectra's reported size is dominated by the embedded *Spectra* copy (3.27 MB); the *Chromatograms* wrapper itself adds only 0.23 MB.

5. Benchmark: Data Access (Intensity Extraction)

How long does each backend take to actually read the intensity data? For *xcms MChromatograms*, the canonical accessor is `lapply(mChr, intensity)`, since `intensity()` does not dispatch on the parent container; this is included for a like-for-like comparison.

```
bench_access <- bench::mark(
  Spectra_intensity = intensity(chr_spectra),
  Memory_intensity  = intensity(chr_mem),
  xcms_lapply       = lapply(chr_xcms, intensity),
  iterations = 5,
  check = FALSE
)

# Peak RAM held during a single intensity extraction call.
peak_access <- peakRAM::peakRAM(
  { intensity(chr_spectra) },
  { intensity(chr_mem) },
  { lapply(chr_xcms, intensity) }
)
peak_access_lookup <- setNames(
  peak_access$Peak_RAM_Used_MiB,
  c("Spectra_intensity", "Memory_intensity", "xcms_lapply")
)

access_df <- bench_access |>
  mutate(
    method      = as.character(expression),
    median_s     = as.numeric(median),
    sd_s         = sapply(time, function(t) sd(as.numeric(t))),
    peak_mb      = peak_access_lookup[as.character(expression)]
  ) |>
  select(method, median_s, sd_s, peak_mb)

knitr::kable(
  access_df |>
    mutate(
      median_ms = round(median_s * 1e3, 2),
      sd_ms     = round(sd_s * 1e3, 2),
      peak_mb   = round(peak_mb, 2)
    ) |>
    select(method, median_ms, sd_ms, peak_mb),
  col.names = c("Method", "Median Time (ms)", "SD (ms)", "Peak RAM (MB)"),
  caption = "Intensity extraction benchmark (5 iterations)"
)
```

Table 7: Intensity extraction benchmark (5 iterations)

Method	Median Time (ms)	SD (ms)	Peak RAM (MB)
Spectra_intensity	2188.36	126.19	393.2
Memory_intensity	0.37	0.14	0.0
xcms_lapply	0.68	0.07	0.1

ChromBackendMemory and *xcms* both return pre-loaded data and access is sub-millisecond, whereas *ChromBackendSpectra* pays an on-the-fly extraction cost on every call - the design trade-off it makes in exchange for its lower memory footprint.

6. Benchmark: Computation via Lazy Processing Queue

A key feature of **Chromatograms** is its **lazy processing queue**: functions added via *addProcessing()* are stored in the object and only executed when peaks data are accessed (e.g. via *intensity()*). This defers computation until the result is needed and batches the whole queue into a single pass. We compare this against the equivalent in *xcms*, where the same operations must be applied manually via *lapply()*.

```
# Function 1: Smoothing with running mean
smooth_chromatogram <- function(object) {
  pd <- object@peaksData
  for (i in seq_along(pd)) {
    int <- pd[[i]]$intensity
    if (length(int) >= 5) {
      for (j in seq_len(3)) {
        int <- caTools::runmean(int, k = 5, endrule = "mean")
      }
      pd[[i]]$intensity <- int
    }
  }
  object@peaksData <- pd
  object
}

# Function 2: Compute derivatives and curvature
compute_derivatives <- function(object) {
  pd <- object@peaksData
  for (i in seq_along(pd)) {
    int <- pd[[i]]$intensity
    if (length(int) >= 3) {
      d1 <- diff(int)
      d2 <- diff(d1)
      d1_padded <- c(0, d1)
      d2_padded <- c(0, 0, d2)
      pd[[i]]$intensity <- int + 0.1 * d1_padded + 0.05 * d2_padded
    }
  }
}
```

```

}
object@peaksData <- pd
object
}

# Function 3: Adaptive baseline correction
correct_baseline <- function(object) {
  pd <- object@peaksData
  for (i in seq_along(pd)) {
    int <- pd[[i]]$intensity
    if (length(int) > 0) {
      baseline <- caTools::runmin(int, k = ceiling(length(int) / 20))
      pd[[i]]$intensity <- int - baseline
    }
  }
  object@peaksData <- pd
  object
}

```

```

chr_s_processed <- chr_spectra |>
  addProcessing(smooth_chromatogram) |>
  addProcessing(compute_derivatives) |>
  addProcessing(correct_baseline)

chr_m_processed <- chr_mem |>
  addProcessing(smooth_chromatogram) |>
  addProcessing(compute_derivatives) |>
  addProcessing(correct_baseline)

bench_lazy <- bench::mark(
  ChromBackendSpectra = intensity(chr_s_processed),
  ChromBackendMemory  = intensity(chr_m_processed),
  iterations = 5,
  check = FALSE
)

peak_lazy <- peakRAM::peakRAM(
  { intensity(chr_s_processed) },
  { intensity(chr_m_processed) }
)

peak_lazy_lookup <- setNames(
  peak_lazy$Peak_RAM_Used_MiB,
  c("ChromBackendSpectra", "ChromBackendMemory")
)

# Equivalent manual processing on xcms MChromatograms
process_xcms <- function(chr_xcms_obj) {

```

```

lapply(chr_xcms_obj, function(chr) {
  int <- intensity(chr)
  if (length(int) >= 5) {
    for (j in seq_len(3)) {
      int <- caTools::runmean(int, k = 5, endrule = "mean")
    }
  }
  if (length(int) >= 3) {
    d1 <- diff(int); d2 <- diff(d1)
    int <- int + 0.1 * c(0, d1) + 0.05 * c(0, 0, d2)
  }
  if (length(int) > 0) {
    baseline <- caTools::runmin(int, k = ceiling(length(int) / 20))
    int <- int - baseline
  }
  int
})
}

bench_xcms_comp <- bench::mark(
  `xcms MChromatograms` = process_xcms(chr_xcms),
  iterations = 5,
  check = FALSE
)

peak_xcms_comp <- peakRAM::peakRAM({ process_xcms(chr_xcms) })
peak_xcms_lookup <- setNames(
  peak_xcms_comp$Peak_RAM_Used_MiB,
  "xcms MChromatograms"
)

peak_comp_lookup <- c(peak_lazy_lookup, peak_xcms_lookup)

comp_df <- bind_rows(
  bench_lazy |> mutate(method = as.character(expression)),
  bench_xcms_comp |> mutate(method = as.character(expression))
) |>
mutate(
  median_s = as.numeric(median),
  sd_s      = sapply(time, function(t) sd(as.numeric(t))),
  peak_mb   = peak_comp_lookup[method]
) |>
select(method, median_s, sd_s, peak_mb)

knitr::kable(
  comp_df |>
  mutate(

```

```

median_ms = round(median_s * 1e3, 2),
sd_ms     = round(sd_s * 1e3, 2),
peak_mb   = round(peak_mb, 2)
) |>
select(method, median_ms, sd_ms, peak_mb),
col.names = c("Method", "Median Time (ms)", "SD (ms)", "Peak RAM (MB)"),
caption = "Computation benchmark: 3 processing functions on 100 EICs"
)

```

Table 8: Computation benchmark: 3 processing functions on 100 EICs

Method	Median Time (ms)	SD (ms)	Peak RAM (MB)
ChromBackendSpectra	2564.83	147.73	393.1
ChromBackendMemory	37.37	0.91	37.3
xcms MChromatograms	37.42	4.02	34.8

ChromBackendMemory and *xcms* show comparable computation times once data are in memory; *ChromBackendSpectra* is dominated by on-the-fly data extraction and is therefore slower for a one-shot full pass. Beyond raw timing, however, the processing queue offers something *xcms MChromatograms* does not: a declarative pipeline that can be inspected, serialised, parallelised via *BiocParallel*, and (most importantly) applied only to the subset of chromatograms eventually requested - the property quantified in Supplementary Material 2.

7. Scaling: Multiple Files

How do the three approaches scale as the number of source files grows? We extract **100 EIC targets per file**, so every approach is given the same total work ($100 \times n_files$ chromatograms). The *xcms chromatogram()* function automatically applies every target across all files in the *MsExperiment*, so the *chromData* table for *Chromatograms* is constructed to match - one row per (target, file) combination.

Note: *ChromBackendMemory* time combines *Chromatograms()* (near-zero; metadata only) and *setBackend()* (full extraction). *ChromBackendSpectra* size includes the embedded *Spectra* copy.

```

n_files_vec <- seq_len(length(mzml_files))
n_eics_per_file <- 100L

scaling_results <- lapply(n_files_vec, function(nf) {
  files <- mzml_files[seq_len(nf)]
  sps_sub <- Spectra(files, source = MsBackendMzR())
  origins_sub <- unique(dataOrigin(sps_sub))

  # Generate 100 EIC targets per file so both approaches extract the same data
  set.seed(123)
  target_mz_scaling <- runif(n_eics_per_file,
                             mz_range[1] + 10, mz_range[2] - 10)
  eic_sub <- do.call(rbind, lapply(origins_sub, function(orig) {
    data.frame(

```



```

    mzMin      = target_mz_scaling - mz_tol_da,
    mzMax      = target_mz_scaling + mz_tol_da,
    rtMin      = rt_range[1],
    rtMax      = rt_range[2],
    msLevel    = 1L,
    dataOrigin = orig
  )
}))

# --- xcms setup (shared across iterations) ---
od_sub <- MsExperiment()
spectra(od_sub) <- sps_sub
abs_sub <- normalizePath(files)
sampleData(od_sub) <- S4Vectors::DataFrame(
  sample = seq_along(abs_sub), file = abs_sub
)
od_sub <- linkSampleData(od_sub,
  with = "sampleData.file = spectra.dataOrigin"
)

# Benchmark all three approaches with 5 iterations each.
# ChromBackendMemory time includes Chromatograms() + setBackend() - the
# full creation cost - so it is benchmarked as a single composite call.
# xcms extracts all EIC targets across all files in the MsExperiment,
# producing n_eics_per_file × nf chromatograms - matching eic_sub above.
bench_res <- bench::mark(
  ChromBackendSpectra = {
    Chromatograms(sps_sub, chromData = eic_sub)
  },
  ChromBackendMemory = {
    setBackend(Chromatograms(sps_sub, chromData = eic_sub),
      ChromBackendMemory())
  },
  `xcms MChromatograms` = {
    chromatogram(
      od_sub,
      mz = cbind(target_mz_scaling - mz_tol_da,
        target_mz_scaling + mz_tol_da),
      rt = cbind(rep(rt_range[1], n_eics_per_file),
        rep(rt_range[2], n_eics_per_file)),
      msLevel = 1L
    )
  },
  iterations = 5,
  check = FALSE
)

```

```

# Object sizes are deterministic - measure once per method.
c_s <- Chromatograms(sps_sub, chromData = eic_sub)
c_m <- setBackend(c_s, ChromBackendMemory())
c_x <- chromatogram(
  od_sub,
  mz = cbind(target_mz_scaling - mz_tol_da,
              target_mz_scaling + mz_tol_da),
  rt = cbind(rep(rt_range[1], n_eics_per_file),
              rep(rt_range[2], n_eics_per_file)),
  msLevel = 1L
)
size_lookup <- c(
  ChromBackendSpectra = as.numeric(object.size(c_s)) / 1024^2,
  ChromBackendMemory = as.numeric(object.size(c_m)) / 1024^2,
  `xcms MChromatograms` = as.numeric(object.size(c_x)) / 1024^2
)

bench_res |>
  mutate(
    method = as.character(expression),
    n_files = nf,
    n_eics = nrow(eic_sub),
    n_spectra = length(sps_sub),
    median_s = as.numeric(median),
    sd_s = sapply(time, function(t) sd(as.numeric(t))),
    size_mb = size_lookup[method]
  ) |>
  select(method, n_files, n_eics, n_spectra, median_s, sd_s, size_mb)
})

scaling_df <- bind_rows(scaling_results)

ggplot(scaling_df,
  aes(x = n_files, y = median_s, colour = method, group = method)) +
  geom_errorbar(aes(ymin = pmax(median_s - sd_s, 1e-9),
                        ymax = median_s + sd_s),
                width = 0.2, linewidth = 0.6, alpha = 0.6) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_colour_methods() +
  labs(
    x = "Number of mzML Files",
    y = "Median Time (seconds)",
    colour = "Method",
    title = "Object Creation Time"
  ) +
  theme_minimal(base_size = 13)

```

```
ggplot(scaling_df,
      aes(x = n_files, y = size_mb, colour = method, group = method)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2.5) +
  scale_colour_methods() +
  labs(
    x = "Number of mzML Files",
    y = "Object Size (MB)",
    colour = "Method",
    title = "RAM Footprint"
  ) +
  theme_minimal(base_size = 13)
```

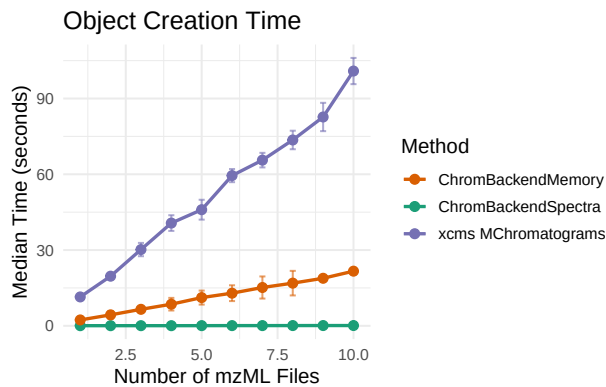


Figure 4: Instantiation time vs number of files. Points and error bars show median and standard deviation across five iterations.

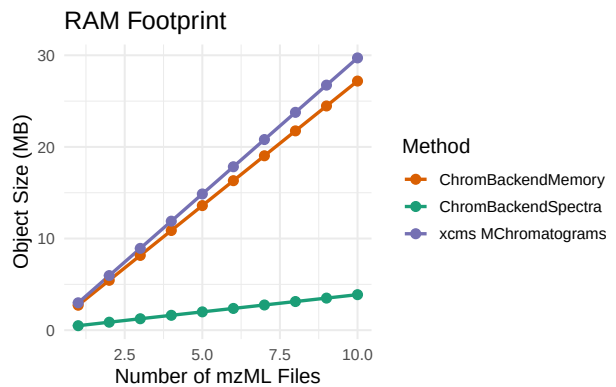


Figure 5: Object size vs number of files

Quantifying scaling directly (multi-file)

The same approach - observed ratio at $n_{\max}$ vs $n_{\min}$, compared with the linear expectation $n_{\max}/n_{\min}$ - quantifies how each method scales with the number of source files.

```
scaling_files_subset <- scaling_df |>
  filter(n_files > 1L, median_s > 0)

nf_min <- min(scaling_files_subset$n_files)
nf_max <- max(scaling_files_subset$n_files)
nf_ratio_expected <- nf_max / nf_min

scaling_ratios_files <- scaling_files_subset |>
  filter(n_files %in% c(nf_min, nf_max)) |>
  select(method, n_files, median_s) |>
  tidyr::pivot_wider(names_from = n_files, values_from = median_s,
                     names_prefix = "t_n") |>
  rename(t_min = paste0("t_n", nf_min),
```

```

      t_max = paste0("t_n", nf_max)) |>
mutate(
  time_ratio      = t_max / t_min,
  linear_ratio    = nf_ratio_expected,
  super_linear_factor = time_ratio / linear_ratio
)

ratio_mem_files <- scaling_ratios_files$time_ratio[
  scaling_ratios_files$method == "ChromBackendMemory"]
ratio_xcms_files <- scaling_ratios_files$time_ratio[
  scaling_ratios_files$method == "xcms MChromatograms"]
sl_mem_files <- scaling_ratios_files$super_linear_factor[
  scaling_ratios_files$method == "ChromBackendMemory"]
sl_xcms_files <- scaling_ratios_files$super_linear_factor[
  scaling_ratios_files$method == "xcms MChromatograms"]

knitr::kable(
  scaling_ratios_files |>
  mutate(across(c(t_min, t_max), \(x) round(x, 3)),
    across(c(time_ratio, super_linear_factor), \(x) round(x, 2))),
  col.names = c("Method",
    paste0("t at n=", nf_min, " (s)"),
    paste0("t at n=", nf_max, " (s)"),
    "Observed ratio",
    "Linear expectation",
    "Super-linear factor"),
  caption = paste0(
    "Multi-file scaling: observed time ratio vs linear expectation (",
    nf_ratio_expected, "x).")
)

```

Table 9: Multi-file scaling: observed time ratio vs linear expectation (5x).

Method	t at n=2 (s)	t at n=10 (s)	Observed ratio	Linear expectation	Super-linear factor
ChromBackendSpectra	0.051	0.109	2.15	5	0.43
ChromBackendMemory	4.335	21.637	4.99	5	1.00
xcms MChro- matograms	19.649	100.876	5.13	5	1.03

For the multi-file benchmark: **ChromBackendMemory** time grows $\approx 4.99\times$ from 2 to 10 files (linear expectation: $5\times$; super-linear factor ≈ 1), whereas **xcms MChromatograms** grows $\approx 5.13\times$ over the same range (super-linear factor ≈ 1).

8. Summary Table

Table 10: Feature comparison of chromatogram backends

Metric	ChromBackendSpectra	ChromBackendMemory	xcms
Instantiation time	Almost immediate (<1 s)	Proportional to data size	Proportional to data size
RAM footprint	Minimal (metadata only)	Full data in RAM	Full data in RAM
Intensity access (first call)	Slower (computed on-the-fly)	Fast (pre-loaded)	Fast (pre-loaded)
Intensity access (repeated)	Same (no caching)	Fast	Fast
Variable-length support	Yes	Yes	No (MChromatograms matrix)
Lazy processing queue	Yes	Yes	No
Backend switching	Yes ($\rightarrow$ Memory)	N/A (target only)	No
Parallel metrics (BiocParallel)	Yes	Yes	No

Discussion

The benchmarks above quantify three trade-offs between the **Chromatograms** backends and *xcms*.

Instantiation scales differently across methods. *ChromBackendSpectra* stores only metadata and stays **essentially constant** with the number of EIC targets (the timing barely changes between $n = 10$ and $n = 1000$ - observed ratio $\approx 1.04\times$ vs the $100\times$ that linear scaling would predict). Both *ChromBackendMemory* (via *setBackend()*) and *xcms chromatogram()* must extract the intensity data immediately, but *setBackend()* scales **roughly linearly** - the observed $0.9\times$ increase from $n = 10$ to $n = 1000$ EIC targets is very close to the linear expectation of $100\times$ (super-linear factor ≈ 0.01). *xcms*, by contrast, takes $\approx 27.37\times$ longer over the same range (super-linear factor ≈ 0.3) - i.e. its instantiation cost grows substantially faster than the work itself, and the gap with *ChromBackendMemory* therefore widens with every additional EIC. The multi-file benchmark shows the same pattern when source files rather than targets are added.

RAM footprint reflects each backend’s design. After creation, the 100-EIC objects are compact: *ChromBackendSpectra* at 3.49 MB (dominated by the embedded **Spectra** copy), *ChromBackendMemory* at 2.72 MB, and *xcms* at 2.99 MB. All three grow roughly linearly with the number of files, but *ChromBackendSpectra* remains the lightest and *xcms* consistently the largest.

Data access and computation expose the immediate/lazy trade-off. *ChromBackendMemory* and *xcms* return pre-loaded intensity vectors in sub-millisecond time; *ChromBackendSpectra* is orders of magnitude slower because each call triggers on-the-fly extraction. The same pattern holds for computation: *ChromBackendMemory* with *addProcessing()* and *xcms* with manual *lapply()* perform comparably, while *ChromBackendSpectra*’s time is dominated by extraction. Crucially, *Chromatograms* adds a declarative processing queue that can be inspected, serialised, and parallelised via **BiocParallel** - and that is applied only to the subset of chromatograms the user actually requests (see Supplementary Material 3).

Recommended workflow. Start with *ChromBackendSpectra* to avoid materialising the full

dataset up-front, queue lazy transformations, subset to the region of interest, and only then call *setBackend(ChromBackendMemory())* on that subset for fast repeated access.

Key results for manuscript

Table 11: Headline numbers used in the main manuscript. Re-check these against the paper after each rendering.

Section	Metric	Value
EIC scaling: absolute times (s)	ChromBackendSpectra at n=10	0.13 s
EIC scaling: absolute times (s)	ChromBackendSpectra at n=1000	0.13 s
EIC scaling: absolute times (s)	ChromBackendMemory at n=10	3.32 s
EIC scaling: absolute times (s)	ChromBackendMemory at n=1000	2.99 s
EIC scaling: absolute times (s)	xcms MChromatograms at n=10	2.84 s
EIC scaling: absolute times (s)	xcms MChromatograms at n=1000	77.62 s
EIC scaling: ratios	ChromBackendSpectra ratio (n_max vs n_min)	1.04x
EIC scaling: ratios	ChromBackendMemory ratio (n_max vs n_min)	0.90x
EIC scaling: ratios	xcms ratio (n_max vs n_min)	27.37x
EIC scaling: ratios	xcms super-linear factor	0.3x
EIC scaling: ratios	Linear expectation	100x
EIC scaling: memory footprint (MB)	ChromBackendSpectra (100-EIC object)	3.49 MB
EIC scaling: memory footprint (MB)	ChromBackendMemory (100-EIC object)	2.72 MB
EIC scaling: memory footprint (MB)	xcms (100-EIC object)	2.99 MB
File scaling: absolute times (s)	ChromBackendMemory at nf=2	4.34 s
File scaling: absolute times (s)	ChromBackendMemory at nf=10	21.64 s
File scaling: absolute times (s)	xcms MChromatograms at nf=2	19.65 s
File scaling: absolute times (s)	xcms MChromatograms at nf=10	100.88 s
File scaling: ratios	ChromBackendMemory file-scaling ratio	4.99x
File scaling: ratios	xcms file-scaling ratio	5.13x
File scaling: ratios	Linear expectation	5x
Benchmark parameters	EIC targets (n_min / n_max)	10 / 1000
Benchmark parameters	Files (nf_min / nf_max)	2 / 10

Session Info

```
sessionInfo()
```

R version 4.6.0 (2026-04-24)

Platform: x86_64-pc-linux-gnu

Running under: Ubuntu 24.04.4 LTS

Matrix products: default

BLAS: /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3

LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblas-p-r0.3.26.so; LAPACK version 3.11.0

locale:

```
[1] LC_CTYPE=en_US.UTF-8      LC_NUMERIC=C
[3] LC_TIME=en_US.UTF-8       LC_COLLATE=en_US.UTF-8
[5] LC_MONETARY=en_US.UTF-8   LC_MESSAGES=en_US.UTF-8
[7] LC_PAPER=en_US.UTF-8      LC_NAME=C
[9] LC_ADDRESS=C              LC_TELEPHONE=C
[11] LC_MEASUREMENT=en_US.UTF-8 LC_IDENTIFICATION=C
```

time zone: Etc/UTC

tzcode source: system (glibc)

attached base packages:

```
[1] stats4      stats      graphics  grDevices  utils      datasets  methods
[8] base
```

other attached packages:

```
[1] tidyr_1.3.2          ggplot2_4.0.3
[3] dplyr_1.2.1          microbenchmark_1.5.0
[5] peakRAM_1.0.2        bench_1.1.4
[7] MsExperiment_1.14.0  xcms_4.10.0
[9] MsBackendMetaboLights_1.7.1 Spectra_1.22.0
[11] S4Vectors_0.50.1     BiocGenerics_0.58.1
[13] generics_0.1.4       Chromatograms_1.2.0
[15] ProtGenerics_1.44.0  BiocParallel_1.46.0
```

loaded via a namespace (and not attached):

```
[1] bitops_1.0-9          DBI_1.3.0
[3] httr2_1.2.2           rlang_1.2.0
[5] magrittr_2.0.5        clue_0.3-68
[7] MassSpecWavelet_1.78.0 otel_0.2.0
[9] matrixStats_1.5.0     compiler_4.6.0
[11] RSQLite_3.53.1        PTMods_1.0.0
[13] vctrs_0.7.3           reshape2_1.4.5
[15] stringr_1.6.0         pkgconfig_2.0.3
[17] MetaboCoreUtils_1.20.1 crayon_1.5.3
[19] fastmap_1.2.0         dbplyr_2.5.2
[21] XVector_0.52.0        labeling_0.4.3
[23] caTools_1.18.3        rmarkdown_2.31
[25] preprocessCore_1.74.0 purrr_1.2.2
[27] bit_4.6.0             xfun_0.57
[29] MultiAssayExperiment_1.38.0 cachem_1.1.0
[31] jsonlite_2.0.0        progress_1.2.3
[33] blob_1.3.0            DelayedArray_0.38.2
[35] parallel_4.6.0        prettyunits_1.2.0
[37] cluster_2.1.8.2       R6_2.6.1
[39] stringi_1.8.7         RColorBrewer_1.1-3
[41] limma_3.68.4          GenomicRanges_1.64.0
[43] iterators_1.0.14      Rcpp_1.1.1-1.1
```

[45] Seqinfo_1.2.0	SummarizedExperiment_1.42.0
[47] knitr_1.51	IRanges_2.46.0
[49] Matrix_1.7-5	igraph_2.3.2
[51] tidyselect_1.2.1	dichromat_2.0-0.1
[53] abind_1.4-8	yaml_2.3.12
[55] doParallel_1.0.17	codetools_0.2-20
[57] affy_1.90.0	curl_7.1.0
[59] lattice_0.22-9	tibble_3.3.1
[61] plyr_1.8.9	withr_3.0.2
[63] Biobase_2.72.0	S7_0.2.2
[65] evaluate_1.0.5	BiocFileCache_3.2.0
[67] pillar_1.11.1	affyio_1.82.0
[69] BiocManager_1.30.27	filelock_1.0.3
[71] MatrixGenerics_1.24.0	foreach_1.5.2
[73] MSnbase_2.37.0	MALDIquant_1.22.3
[75] ncd4_1.24	hms_1.1.4
[77] scales_1.4.0	glue_1.8.1
[79] MsFeatures_1.20.0	lazyeval_0.2.3
[81] tools_4.6.0	mzID_1.50.0
[83] data.table_1.18.4	QFeatures_1.22.0
[85] vsn_3.80.0	mzR_2.46.0
[87] XML_3.99-0.23	fs_2.1.0
[89] grid_4.6.0	impute_1.86.0
[91] MsCoreUtils_1.24.0	PSMatch_1.16.0
[93] cli_3.6.6	rappdirs_0.3.4
[95] profmem_0.7.0	S4Arrays_1.12.0
[97] AnnotationFilter_1.36.0	pcaMethods_2.4.0
[99] gtable_0.3.6	digest_0.6.39
[101] SparseArray_1.12.2	farver_2.1.2
[103] memoise_2.0.1	htmltools_0.5.9
[105] lifecycle_1.0.5	statmod_1.5.2
[107] bit64_4.8.2	MASS_7.3-65